

```
devops@linux: ~  
  
devops@linux:~$ whoami  
devops  
  
devops@linux:~$ ls -l deploy.sh  
-rwxr-xr-- 1 devops team 2048 deploy.sh  
  
devops@linux:~$ sudo systemctl status nginx  
Active: active (running)  
  
devops@linux:~$ ps aux | grep nginx  
www-data 1243 nginx: worker  
  
devops@linux:~$ _
```



Linux for DevOps

Commands, concepts and real-world practice,
explained simply with visuals.

CREATED BY



Sharad Verma

@sharadverma638

Commands

Vim

Permissions

Users

Processes

Networking

systemd

13

topics

20+

illustrations

300+

commands



Want to learn DevOps?



github.com/sharadverma638/sv-opslibrary

Contents

13 sections. Read in order, or jump to what you need.

01	What is Linux? The operating system behind servers, cloud and DevOps	2
02	Linux Basics: Quick Reference Directories, commands, permissions and more	3
03	CRUD Operations Create, Read, Update, Delete	5
04	Vim Editor Modes and editing operations	7
05	File & Directory Permissions Who can read, write and run what	10
06	User Management Users, groups and sudo	13
07	Process Management See, control and prioritise running programs	16
08	Package Management Install, update and remove software	19
09	Service Management systemd, init and daemons	22
10	Network Management IP, DNS, ports and firewalls	25
11	Troubleshooting Find and fix problems step by step	29
12	Practical: User Access Control Full sudo vs limited command access	33
13	File System Structure Everything starts at /	36



Suggested study path

01

What is Linux?

The operating system behind servers, cloud and DevOps

Linux is an **Operating System (OS)**, like Windows or macOS. But unlike them, Linux is **open-source**: it is free and its code is publicly available.

Linux is used in **servers, cloud computing, mobile (Android), embedded systems, and DevOps tools**.

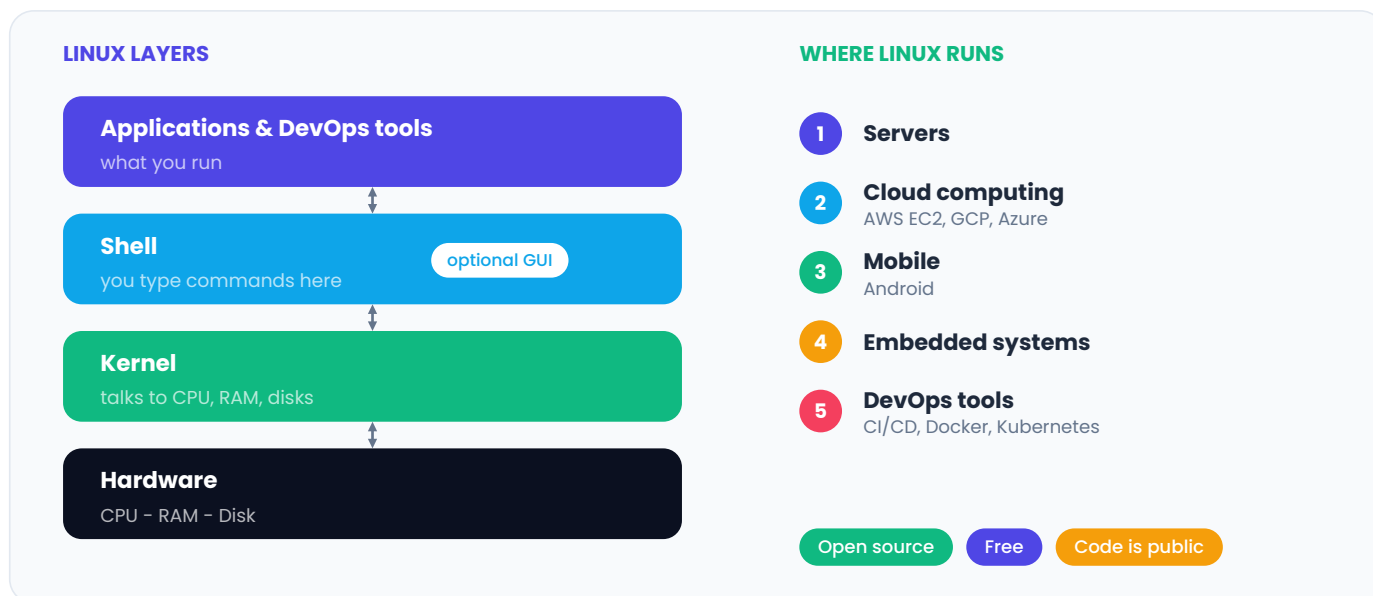


Fig 1. The layers of a Linux system and where Linux runs.

Basic components of the Linux OS

Component	Explanation
Kernel	The core part of Linux that talks to hardware (like CPU, RAM, disks).
Shell	A program that lets you interact with the OS via commands.
File System	Organizes and stores data. Everything in Linux is treated as a file (even devices).
GUI	Optional. Linux can have graphical interfaces like GNOME or KDE, but is often used via terminal.

How it helps in DevOps

Linux is the foundation of cloud servers (AWS EC2, GCP, Azure), CI/CD pipelines, Docker & Kubernetes, and automation tools like Ansible and Terraform.

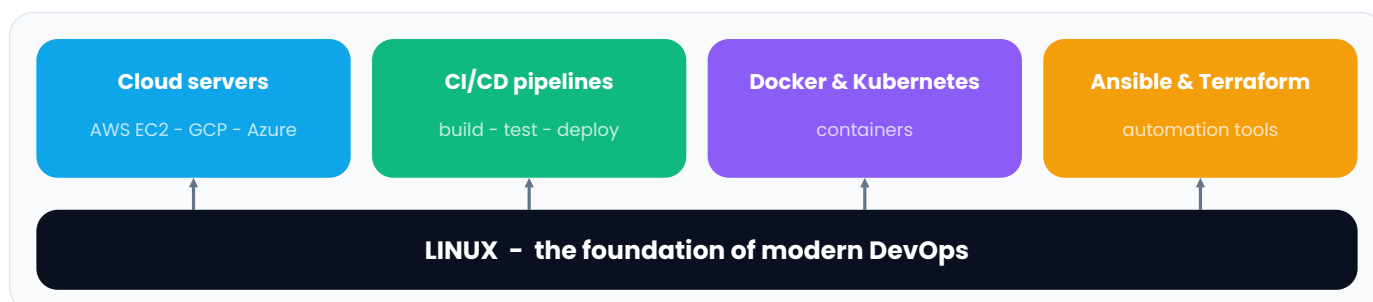


Fig 2. Most DevOps tools stand on top of Linux.

02

Linux Basics: Quick Reference

Directories, commands, permissions and more



A quick tour of the essentials. Each topic gets its own full section later in these notes.

Linux directory structure

Linux starts with the **root (/)** directory. Below it, there are folders with fixed purposes:

Directory	Purpose
/	Root of the file system
/home	User folders (like C:\Users)
/bin	Essential command binaries (like ls, cp, mv)
/etc	System config files
/var	Variable files (like logs)
/tmp	Temporary files
/usr	User programs and libraries
/root	Home directory of the root (admin) user
/dev	Device files (like USB, hard disks)
/proc	System and process info (virtual folder)

Basic Linux commands

Command	Use
pwd	Show current directory
ls	List files and folders
cd	Change directory
mkdir	Make a directory
touch	Create an empty file
cp	Copy files/folders
mv	Move or rename files
rm	Delete files
cat	Show file contents
clear	Clear the terminal
exit	Close the terminal or shell

Users and permissions

Linux is **multi-user**. Every file and folder has permissions and ownership.

Users	Meaning
root	Superuser (admin)
Normal users	Limited access

Each file has 3 types of permissions for **Owner**, **Group** and **Others**. Permission types: **r** = read, **w** = write, **x** = execute.

```
-rwxr-xr-- 1 user group 1234 Jul 1 file.sh
```

Part	Meaning
rwx (owner)	full permission
r-x (group)	read & execute
r-- (others)	read only

Package management (installing software)

Distro family	Command
Ubuntu/Debian	<code>apt-get install <package-name></code>
Red Hat/CentOS	<code>yum install <package-name></code> or <code>dnf install</code>
Others	May use pacman, zypper, snap, flatpak

Processes and system monitoring

Command	Description
<code>ps</code>	Show running processes
<code>top</code>	Live view of system usage (CPU, memory)
<code>kill</code>	Stop a process
<code>htop</code>	Better version of top (if installed)

File compression and archiving

Command	Description
<code>tar -cvf</code>	Create archive
<code>tar -xvf</code>	Extract archive
<code>gzip, gunzip</code>	Compress and decompress files



Fig 3. Bundle with tar, shrink with gzip.

Important concepts

Concept	Meaning
Shell Script	A file with commands you can execute like a program (<code>.sh</code>)
Cron Job	Schedule a task to run automatically
Log files	Located in <code>/var/log</code> , useful for troubleshooting
SSH	Remote login to a Linux machine (<code>ssh user@ip</code>)
Sudo	Temporarily run commands as root user (<code>sudo <command></code>)

03

CRUD Operations

Create, Read, Update, Delete



CRUD stands for **Create, Read, Update, Delete**. In Linux, CRUD operations can be performed on **files, directories, file content, and system records** (like user data or processes).



Fig 4. CRUD operations in Linux at a glance.

C: Create operations

Task	Command
Create a file	<code>touch filename.txt</code> Example: <code>touch notes.txt</code>
Create a file with content	<code>echo "Hello, Linux!" > hello.txt</code>
...or type the content yourself	<code>cat > data.txt</code> , type your content, press Ctrl+D to save
Create a directory (folder)	<code>mkdir foldername</code> Example: <code>mkdir projects</code>
Create nested directories	<code>mkdir -p devops/scripts/yaml</code>

R: Read operations

Task	Command
View file content	<code>cat filename.txt</code>
Scrollable view	<code>less filename.txt</code>
Page-wise output	<code>more filename.txt</code>
First 10 lines / last 10 lines	<code>head filename.txt</code> / <code>tail filename.txt</code>
Read a directory	<code>ls</code> list files, <code>ls -l</code> detailed view, <code>ls -a</code> show hidden files
Read a specific line (sed or awk)	<code>sed -n '3p' filename.txt</code> show 3rd line; <code>awk 'NR==3' filename.txt</code> another method

U: Update operations

Task	Command
Append to file	<code>echo "New line" >> file.txt</code>
Edit file (editors)	<code>nano filename.txt</code> beginner-friendly terminal editor; <code>vim filename.txt</code> powerful but advanced editor
Replace text in file	<code>sed -i 's/oldtext/newtext/g' filename.txt</code>
Example	<code>sed -i 's/Devops/Linux DevOps/g' notes.txt</code>
Rename file or folder	<code>mv oldname.txt newname.txt</code>
Move file to different location	<code>mv file.txt /home/user/docs/</code>

D: Delete operations

Task	Command
Delete a file	<code>rm filename.txt</code>
Delete a directory	<code>rm -r foldername</code>
Force delete	<code>rm -rf foldername</code>

WATCH OUT

Be careful with `rm -rf /`: it can wipe out the entire OS.

Bonus: real-time DevOps use cases of CRUD in Linux

Linux command	Task
<code>touch nginx.conf</code>	Create a config file
<code>less /var/log/syslog</code>	Read server logs
<code>nano deployment.yaml</code>	Update YAML for deployment
<code>rm -rf /tmp/*</code>	Delete temporary files
<code>mv backup.tar.gz /mnt/backup/</code>	Move backup

Sample exercise to practice

```
# Step 1: Create a directory and file
mkdir devops_practice
cd devops_practice
touch inventory.txt
# Step 2: Add data
echo "Server1: 192.168.1.1" >> inventory.txt
echo "Server2: 192.168.1.2" >> inventory.txt
# Step 3: Read and edit
cat inventory.txt
sed -i 's/192.168.1.2/10.0.0.2/' inventory.txt
# Step 4: Delete the file
rm inventory.txt
```

04

Vim Editor

Modes and editing operations

Vim (Vi Improved) is a **modal text editor** in Linux. It is used for editing config files, shell scripts, YAML, Docker, Terraform and more, especially on headless servers and DevOps workflows. It runs in the terminal and is lightweight, powerful, and scriptable.

Vim has 3 primary modes

Mode	Purpose	Enter mode	Exit mode
1. Normal Mode	Default mode: navigation, deleting, copying	Open Vim or press Esc	Press i, v, or :
2. Insert Mode	Typing text (like in Notepad)	Press i, l, a, A, o, O	Press Esc
3. Command-Line Mode	Save, exit, search, run commands	Press : from Normal Mode	Press Enter or Esc

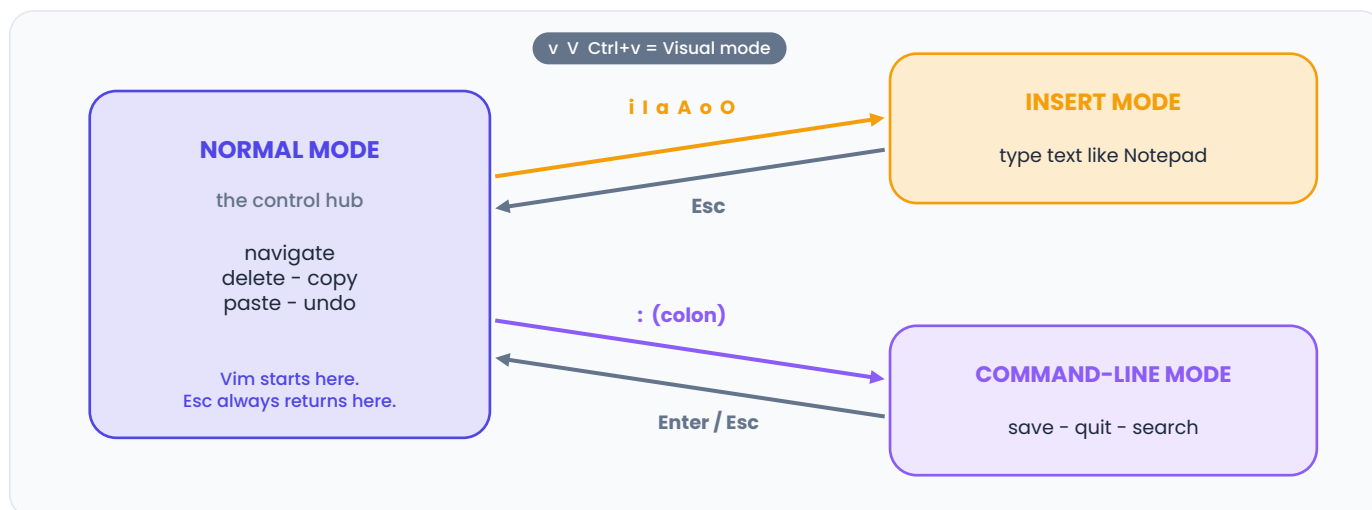


Fig 5. How you move between Vim modes.

1. Normal mode: the control hub

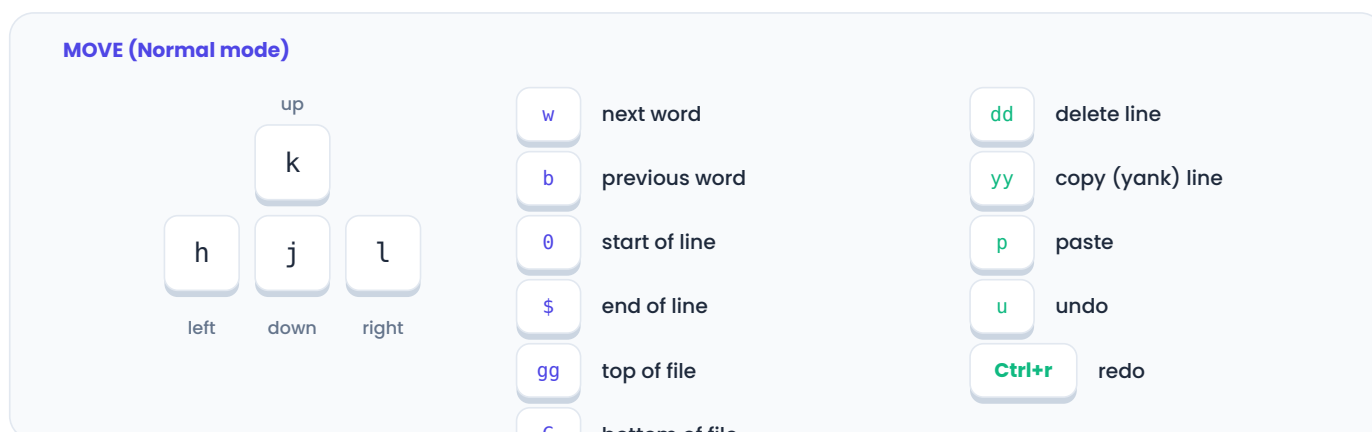


Fig 6. Keys you use in Normal mode.

2. Insert mode: for writing or editing text

Key	Function
i	Insert before cursor
I	Insert at start of line
a	Insert after cursor
A	Insert at end of line
o	Open a new line below
O	Open a new line above

Once in insert mode, you type normally like any text editor. To exit insert mode, press **Esc** to go back to Normal mode.

3. Command-line mode: save, quit, search

From Normal mode, press **:** (colon) to enter command mode.

Command	Meaning
:w	Save file
:q	Quit
:wq or ZZ	Save and quit
:q!	Force quit without saving
:x	Save and exit (if changes made)
:set nu	Show line numbers
:set nonu	Hide line numbers

Editing operations in Vim (DevOps oriented)

File & config editing

```
vim /etc/nginx/nginx.conf
```

1. Navigate to the line using arrow keys or j/k.
2. Press **i** to enter insert mode.
3. Make changes.
4. Press **Esc**, then type **:wq** to save and exit.

Searching and replacing

Task	Command
Search for a word	/nginx, then press n (next match) or N (previous match)
Replace all occurrences in file	:%s/oldtext/newtext/g

Copy, cut, paste (yank, delete, put)

Action	Key
Copy (line)	yy
Copy (3 lines)	3yy
Delete (cut) line	dd
Delete 5 lines	5dd
Paste below	p
Paste above	P

Line and block editing

- **Visual Mode (v):** select characters/words
- **Visual Line Mode (V):** select entire lines
- **Visual Block Mode (Ctrl + v):** select columns
- Then you can press **y**, **d**, or **p** to copy, cut, or paste.

Practical DevOps examples

Task	Vim command
Edit crontab	<code>crontab -e</code> (uses Vim by default)
Change pod name in YAML	<code>vim pod.yaml, /name, i, edit</code>
Format Terraform code	<code>vim main.tf, use =G to auto-indent</code>
Edit Dockerfile	<code>vim Dockerfile</code>
Restart NGINX config	Edit <code>nginx.conf</code> then <code>sudo systemctl restart nginx</code>

Sample practice

```
vim test.sh
```

1. Press **i**, type: `#!/bin/bash` and `echo "Hello, Vim World!"`
2. Press **Esc**
3. Save: `:wq`
4. Make executable: `chmod +x test.sh`
5. Run it: `./test.sh`

Vim cheat sheet summary

Mode	Enter	Purpose	Exit
Normal	Open Vim / Esc	Navigation, editing	i, :, v
Insert	i, a, o, etc.	Typing text	Esc
Command	:	Save, quit, search	Enter or Esc

05

File & Directory Permissions

Who can read, write and run what



Why permissions matter

- Linux is a multi-user OS, and permissions protect files, scripts, and system settings from unauthorized access or modification.
- Common in DevOps, cloud environments, and CI/CD pipelines where secure access is critical.

Permission types

Each file or directory in Linux has 3 types of permissions for 3 types of users.

Symbol	User type	Symbol	Permission	Meaning
u	User (Owner)	r	Read	View content
g	Group	w	Write	Modify content
o	Others (everyone else)	x	Execute	Run file (scripts, binaries) or enter directory

Example output from ls -l

```
-rwxr-xr-- 1 devops team 2048 Jul 1 deploy.sh
```



Fig 7. Reading a permission string.

Part	Meaning
-	File type (- = file, d = directory)
rwx	Owner: read, write, execute
r-x	Group: read, execute
r--	Others: read only
devops	Owner username
team	Group name

Numeric (octal) permissions

Permission	Binary	Octal	rwx combo	Value
r	100	4	rwx	7
w	010	2	rw-	6
x	001	1	r--	4
			r-x	5

Add values to get the final permission. So `chmod 755 filename` means `rwxr-xr-x`.

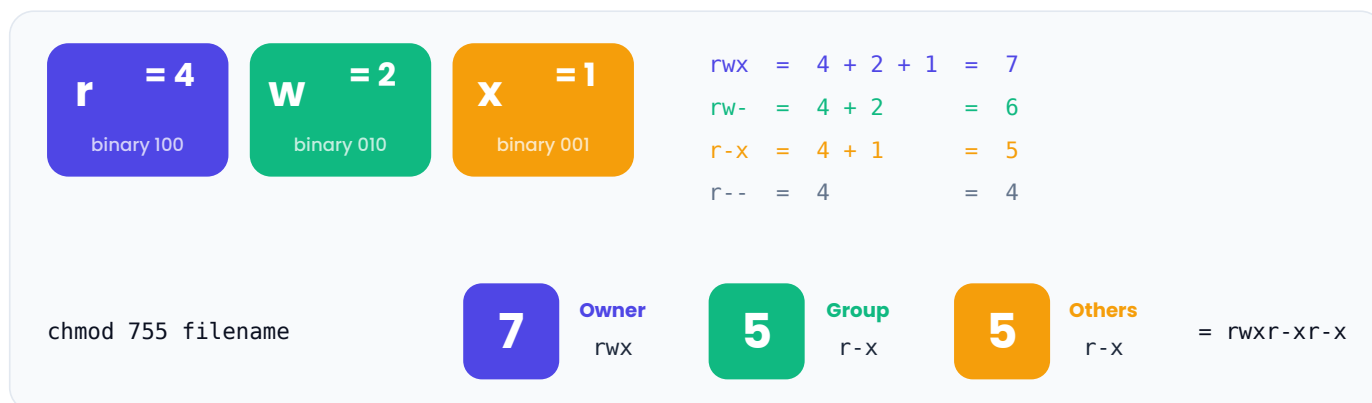


Fig 8. Octal math: add r=4, w=2, x=1.

chmod: change permissions

Symbolic method

```

chmod u+x script.sh # Add execute to user
chmod g-w file.txt  # Remove write from group
chmod o=r file.txt  # Others: read-only

```

Numeric (octal) method

```

chmod 644 file.txt # rw-r--r--
chmod 755 script.sh # rwxr-xr-x
chmod 700 secrets.txt # rwx----- (only owner can access)

```

chown: change ownership

```
chown user:group file
```

Example: `chown devops:team deploy.sh`

Permissions for directories

Permission	Effect
r	Can list files (<code>ls</code>)
w	Can add/remove files
x	Can enter directory (<code>cd</code>)

Example: `drwxr-x--- 2 riyas team 4096 Jul 1 logs/`

- **Owner:** full access
- **Group:** can read and enter
- **Others:** no access

Special permissions

Symbol	Name	Use
s	Setuid / Setgid	Run program with owner/group privileges
t	Sticky Bit	Used in /tmp so only file owner can delete their files

```
chmod +t /shared/folder
ls -ld /shared/folder
# drwxrwxrwt -> sticky bit set
```

DevOps use cases

Task	Related permission
Make shell script executable	<code>chmod +x script.sh</code>
Restrict .env to user only	<code>chmod 600 .env</code>
Secure private keys	<code>chmod 400 key.pem</code>
Allow group to deploy	<code>chown :deployers deploy.sh</code>
Lock config file	<code>chmod 444 config.yml (read-only)</code>

Summary table

Octal	Symbolic	Meaning
777	<code>rw-rw-rw-</code>	Everyone full access
755	<code>rw-r-xr-x</code>	Owner full, others can read/execute
700	<code>rw-x-----</code>	Only owner access
644	<code>rw-r--r--</code>	Read/write owner, read others
600	<code>rw-----</code>	Only owner can read/write

Practice exercise

```
# Create a file
touch test.sh
# Give only the owner full access
chmod 700 test.sh
# Make it executable
chmod +x test.sh
# Change ownership to "devops" user and "ci" group
chown devops:ci test.sh
# Set read-only for everyone
chmod 444 test.sh
```

06

User Management

Users, groups and sudo



Why is user management important?

In a multi-user system (like Linux servers or cloud environments), managing users, groups, and permissions ensures:

- **Security & isolation**
- **Controlled access** to files/scripts/services
- **Auditing and accountability**

1. Understanding users & groups

User type	Description
Root	Superuser with all permissions
System Users	Created by services (e.g., nginx, mysql)
Regular Users	Created by admins for login and tasks



Fig 9. Users, groups and the files that store them.

- Groups are collections of users.
- Users can belong to multiple groups.
- Groups simplify permission management (e.g., give access to a folder for all devops users).

2. Create, delete and manage users

Create user

```
sudo adduser riyas
```

Adds the user with a home directory **/home/riyas** and prompts to set a password. Or the basic (less interactive) way:

```
sudo useradd -m riyas
sudo passwd riyas
```

Delete user

```
sudo deluser riyas          # Keeps home dir
sudo deluser --remove-home riyas
# Or:
sudo userdel -r riyas
```

Modify user

```
sudo usermod -aG devops riyas    # Add to 'devops' group
sudo usermod -l newname oldname  # Rename user
sudo usermod -d /new/home/path riyas # Change home dir
```

3. User info and files

File	Purpose
/etc/passwd	User account info (username, UID, GID, shell)
/etc/shadow	Encrypted passwords
/etc/group	Group definitions
/etc/sudoers	Who can use sudo

View user info

```
cat /etc/passwd | grep riyas
id riyas          # Show UID, GID, groups
groups riyas      # Show group memberships
```

4. Group management

Task	Command
Create group	<code>sudo groupadd devops</code>
Add user to group	<code>sudo usermod -aG devops riyas</code>
Remove user from group (manually)	<code>sudo gpasswd -d riyas devops</code>
Delete group	<code>sudo groupdel devops</code>

5. Sudo (superuser do) access

Grant sudo access to a user

```
sudo usermod -aG sudo riyas
```

Or edit the sudoers file safely with `sudo visudo` and add:

```
riyas ALL=(ALL:ALL) ALL
```

Limited sudo (for security)

Only allow specific commands:

```
riyas ALL=(ALL) NOPASSWD: /bin/systemctl restart nginx
```

6. Home directory & shell

Task	Command
View home	<code>echo \$HOME</code>
Change default shell	<code>chsh -s /bin/bash riyas</code>
Set home manually	<code>usermod -d /custom/home riyas</code>

7. Lock and unlock users

Task	Command
Lock account	<code>sudo usermod -L riyas</code>
Unlock account	<code>sudo usermod -U riyas</code>
Disable login shell (block login)	<code>sudo usermod -s /usr/sbin/nologin riyas</code>

8. Real DevOps use cases

Task	Command / approach
Create a user for Jenkins agent	<code>adduser jenkins_agent</code>
Restrict a user to deploy only	<code>usermod -aG docker deployer</code>
Setup SSH-only user	Create user, disable password, set up SSH key
Group-level permission for /var/www	Create webdev group, set directory group ownership
Give NGINX team restart permission only	<code>sudo visudo</code> + command restriction
SFTP-only access	Create chrooted SFTP users with nologin shell

Cheat sheet summary

Task	Command
Add user	<code>adduser <name></code>
Delete user	<code>deluser <name></code> Or <code>userdel -r</code>
Add to group	<code>usermod -aG <group> <user></code>
List groups	<code>groups <user></code>
Create group	<code>groupadd <group></code>
Lock account	<code>usermod -L <user></code>
Give sudo access	<code>usermod -aG sudo <user></code>

07

Process Management

See, control and prioritise running programs

A **process** is any program or command that is running on your Linux system. It has a unique **PID (Process ID)** and can be running, sleeping, stopped, or zombie.

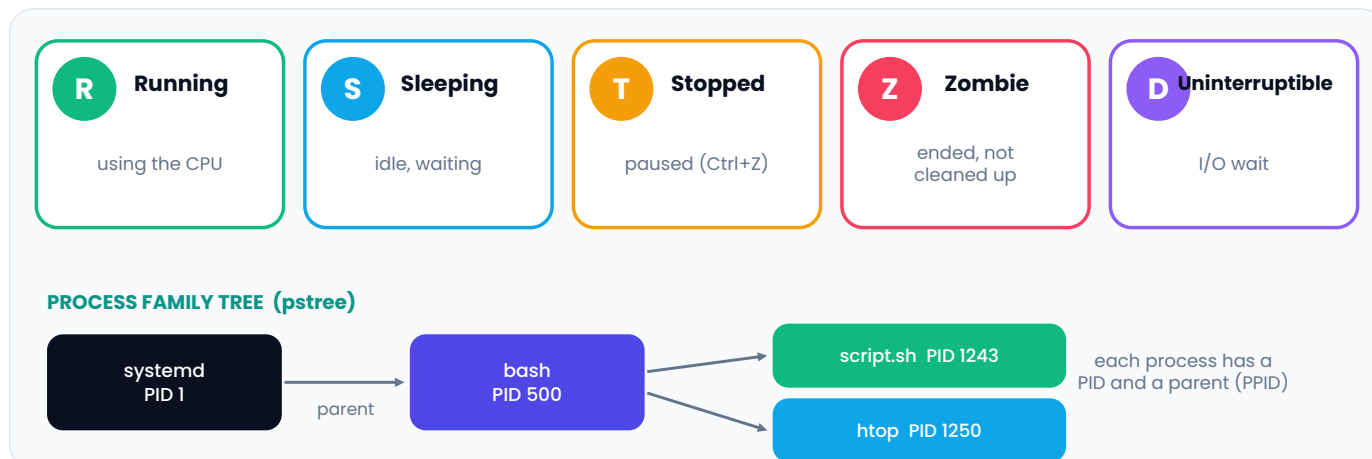


Fig 10. Process states and the parent-child family tree.

State	Meaning
R	Running
S	Sleeping (idle but waiting)
T	Stopped
Z	Zombie (terminated but not cleaned up)
D	Uninterruptible sleep (I/O wait)

1. View running processes

Task	Command
List all processes	<code>ps -ef</code> shows user, PID, parent PID (PPID), time, and command
Real-time monitoring	<code>top</code> interactive, live view of CPU, memory, and processes
Better version of top	<code>htop</code> (requires installation)

```
sudo apt install htop # Debian/Ubuntu
sudo yum install htop # RHEL/CentOS
```

2. Search for a specific process

```
ps aux | grep nginx
pgrep nginx # Show PID(s) of nginx
```

3. Understand process hierarchy

`pstree` visualizes the parent-child process structure.

4. Start a process

Task	Command
Run a script	<code>./script.sh</code>
Run in background	<code>./script.sh &</code>
Keep running after logout (nohup)	<code>nohup ./script.sh &</code>

5. Kill / stop / manage a process

Action	Command
Kill by PID	<code>kill <PID></code>
Force kill	<code>kill -9 <PID></code>
Kill by name	<code>pkill nginx</code>
Kill all matching	<code>killall nginx</code>

Example:

```
kill -9 1243
pkill -f python
```

6. Foreground vs background jobs



Fig 11. Moving a job between foreground and background.

Task	Command
Run in foreground	<code>./longtask.sh</code>
Move to background	Press <code>Ctrl + Z</code> (pauses task), then run <code>bg</code> (resumes in background)
View jobs	<code>jobs</code>
Bring back to foreground	<code>fg %1</code>

7. Set priority (nice, renice)

Task	Command
Launch with low priority	<code>nice -n 10 ./task.sh</code>
Change priority of running process	<code>renice -n 5 -p 1234</code>

- Lower value = higher priority
- **-20** (highest), **19** (lowest)

8. Process details & memory/CPU stats

Task	Command
Check memory usage	<code>ps aux --sort=-%mem head</code>
Check CPU usage	<code>ps aux --sort=-%cpu head</code>

9. DevOps use cases

Task	Command
Restart Jenkins if not running	<code>pgrep jenkins sudo systemctl restart jenkins</code>
Monitor Docker CPU usage	<code>top, htop, or docker stats</code>
Check stuck process during deploy	<code>ps aux grep <name></code>
Kill zombie Python script	<code>kill -9 \$(pgrep -f script.py)</code>
Auto-restart failed process	Use supervisord or systemd

10. Advanced tools

Tool	Use
<code>strace</code>	Trace system calls of a process
<code>lsuf</code>	List files used by processes
<code>watch</code>	Re-run a command repeatedly
<code>systemctl</code>	Manage background services (systemd)
<code>cron</code>	Schedule recurring tasks (not real-time process management but related)

Summary table

Command	Purpose
<code>ps -ef</code>	List all processes
<code>top / htop</code>	Live process monitor
<code>kill, pkill, killall</code>	Stop processes
<code>jobs, fg, bg</code>	Manage background/foreground jobs
<code>nice, renice</code>	Adjust process priority
<code>pgrep, pstree</code>	Search or view hierarchy
<code>lsuf, strace</code>	Debug processes/files

Practice scenario

```
# 1. Create a test script
echo -e "#!/bin/bash\nsleep 300" > sleeper.sh
chmod +x sleeper.sh
# 2. Run it in background
./sleeper.sh &
# 3. View process
ps -ef | grep sleeper
# 4. Kill it
kill -9 <PID>
```

08

Package Management

Install, update and remove software



Packages are compressed files that contain programs, libraries, or tools. **Package managers** are tools to install, update, remove, or search for software on Linux. DevOps often relies on package management to automate infrastructure setup (e.g., installing Docker, Ansible, or monitoring tools).

Types of Linux package managers

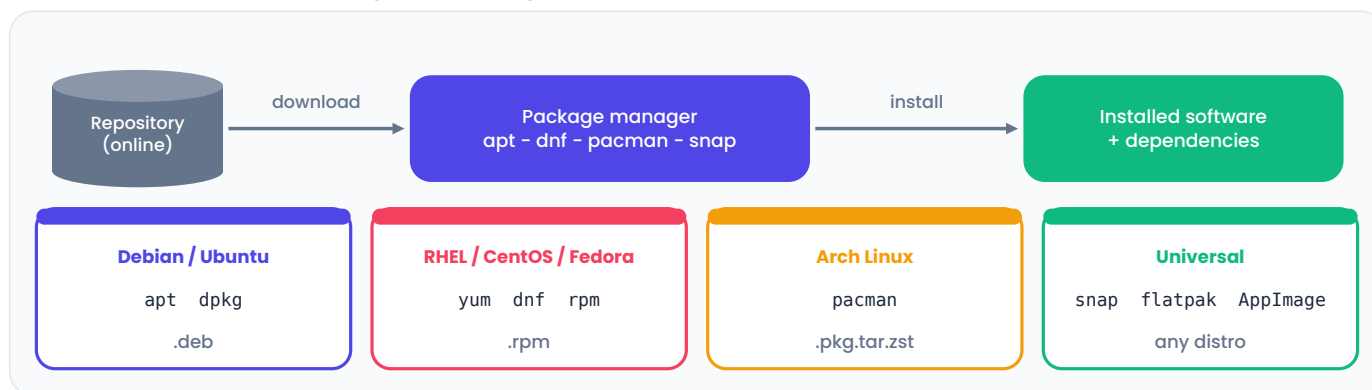


Fig 12. Package managers by distro family.

Distro family	Tool	File extension
Debian/Ubuntu	apt, dpkg	.deb
RHEL/CentOS/Fedora	yum, dnf, rpm	.rpm
Arch Linux	pacman	.pkg.tar.zst
Universal	snap, flatpak, AppImage	-

APT (Advanced Packaging Tool): Ubuntu/Debian

Task	Command
Update list of packages	<code>sudo apt update</code>
Upgrade all packages	<code>sudo apt upgrade</code>
Install package	<code>sudo apt install nginx</code>
Remove package	<code>sudo apt remove nginx</code>
Search package	<code>apt search <name></code>
Get info	<code>apt show <package></code>
Clean old packages	<code>sudo apt autoremove</code>

Install from .deb file

```

sudo dpkg -i file.deb
sudo apt -f install # Fix missing dependencies

```

YUM/DNF: RHEL, CentOS, Fedora

DNF is the newer replacement for YUM.

Task	YUM	DNF
Install	<code>sudo yum install nginx</code>	<code>sudo dnf install nginx</code>
Remove	<code>sudo yum remove nginx</code>	<code>sudo dnf remove nginx</code>
Update system	<code>sudo yum update</code>	<code>sudo dnf upgrade</code>
Search	<code>yum search nginx</code>	<code>dnf search nginx</code>

Install from .rpm

```
sudo rpm -ivh package.rpm
```

Use `yum localinstall package.rpm` to auto-resolve dependencies.

pacman: Arch Linux & derivatives

```
sudo pacman -Syu           # Full system update
sudo pacman -S package-name # Install
sudo pacman -R package-name # Remove
sudo pacman -Ss search-term # Search
```

snap: universal package manager

- Works across distros.
- Packages include all dependencies (like containers).
- Slower but very easy to use.

```
sudo snap install code --classic
sudo snap remove code
snap list
```

flatpak: alternative universal manager

```
flatpak install flathub com.spotify.Client
flatpak run com.spotify.Client
```

DevOps use cases

Task	Command
Install Docker on Ubuntu	<code>sudo apt install docker.io</code>
Install Git on CentOS	<code>sudo yum install git</code>
Setup Prometheus via RPM	<code>sudo rpm -ivh prometheus.rpm</code>
Install Terraform via .deb	<code>sudo dpkg -i terraform_1.7.deb</code>
Automate installs in shell scripts	Include apt, yum, or snap commands
Use Ansible to manage packages	<code>apt: name=nginx state=present</code>

Bonus tools

Tool	Use
<code>dpkg</code>	Low-level Debian tool (for .deb)
<code>rpm</code>	Low-level Red Hat tool (for .rpm)
<code>aptitude</code>	TUI for apt package management
<code>brew</code>	Popular on macOS and Linux (Homebrew)
<code>conda</code> / <code>pip</code>	Python packages (for ML/dev tasks)

Sample practice task (Ubuntu)

```
# 1. Update and install
sudo apt update
sudo apt install htop
# 2. Check installed version
htop --version
# 3. Remove it
sudo apt remove htop
```

Summary table

Action	APT	YUM/DNF	RPM	Snap
Install	<code>apt install</code>	<code>yum install</code>	<code>rpm -ivh</code>	<code>snap install</code>
Remove	<code>apt remove</code>	<code>yum remove</code>	<code>rpm -e</code>	<code>snap remove</code>
Update system	<code>apt upgrade</code>	<code>yum update</code>	-	-
List installed	<code>dpkg -l</code>	<code>yum list installed</code>	<code>rpm -qa</code>	<code>snap list</code>

09

Service Management

systemd, init and daemons



What is a service?

A **service** (also called a **daemon**) is a background process, like nginx, docker, sshd, mysql, etc. Services start **manually** (on demand) or **automatically** (at boot).

In Linux, services are managed using init systems: **older SysVinit** (commands: `service`, `chkconfig`) and **modern systemd** (commands: `systemctl`, `journalctl`).

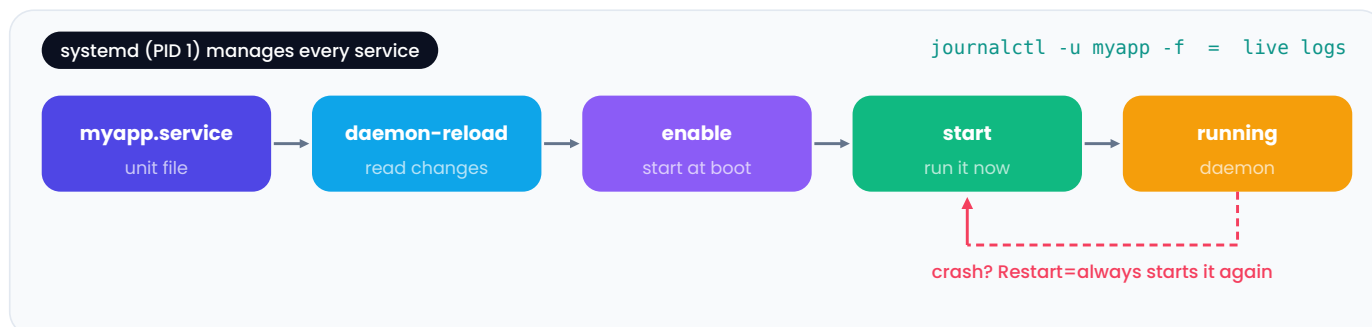


Fig 13. The life of a systemd service.

1. systemctl: the modern service manager (systemd-based)

Most modern Linux distributions (Ubuntu 15+, CentOS 7+, Debian 8+, RHEL 7+) use systemd.

Task	Command
Start a service	<code>sudo systemctl start nginx</code>
Stop a service	<code>sudo systemctl stop nginx</code>
Restart a service	<code>sudo systemctl restart nginx</code>
Reload without restart	<code>sudo systemctl reload nginx</code>
Check status	<code>systemctl status nginx</code>

2. Enable / disable at boot

Action	Command
Enable auto-start	<code>sudo systemctl enable nginx</code>
Disable auto-start	<code>sudo systemctl disable nginx</code>
Check if enabled	<code>systemctl is-enabled nginx</code>

3. View service logs

```
journalctl -u nginx
```

Option	Description
<code>-u</code>	Show logs for specific service
<code>-f</code>	Follow logs (like <code>tail -f</code>)
<code>--since "10 min ago"</code>	Filter by time

Example: `journalctl -u docker -f`

4. Service files and paths (systemd)

Location	Purpose
/etc/systemd/system/	Custom service unit files
/lib/systemd/system/	Default service unit files
*.service	Files that define how to start/stop a service

5. Anatomy of a .service file

Example: /etc/systemd/system/myapp.service

```
[Unit]
Description=My Flask App
After=network.target

[Service]
ExecStart=/usr/bin/python3 /opt/myapp/app.py
WorkingDirectory=/opt/myapp
Restart=always
User=ubuntu

[Install]
WantedBy=multi-user.target
```

Enable and start it:

```
sudo systemctl daemon-reload
sudo systemctl enable myapp
sudo systemctl start myapp
```

6. Legacy commands (SysVinit-based distros)

If your system uses SysVinit (e.g., CentOS 6):

Task	Command
Start	sudo service nginx start
Stop	sudo service nginx stop
Restart	sudo service nginx restart
Enable at boot	chkconfig nginx on
Disable	chkconfig nginx off

7. DevOps real-world use cases

Scenario	Command
Restart Jenkins after update	systemctl restart jenkins
Enable Docker at boot	systemctl enable docker
Check failed services	systemctl --failed
View service logs after crash	journalctl -xe
Set up custom monitoring service	Write .service file + enable it
Auto-restart failed app	Add Restart=always in .service file

Summary: systemctl cheat sheet

Command	Use
<code>systemctl start <service></code>	Start service now
<code>systemctl stop <service></code>	Stop it
<code>systemctl restart <service></code>	Restart it
<code>systemctl status <service></code>	Show status
<code>systemctl enable <service></code>	Start on boot
<code>systemctl disable <service></code>	Don't start on boot
<code>journalctl -u <service></code>	View logs
<code>systemctl daemon-reexec</code>	Reload systemd itself

Best practices (DevOps)

- Use **systemctl** over **service** on modern distros.
- For scripts, check service status before acting (see below).
- Set **Restart** policies in .service files for resilience.
- Always **daemon-reload** after editing service files.

```
if systemctl is-active --quiet nginx; then
    echo "Nginx is running"
fi
```

10

Network Management

IP, DNS, ports and firewalls



Why network management matters

In Linux-based systems (servers, containers, VMs), network management helps to:

- Configure IP addresses, DNS, routing
- Check connectivity and troubleshoot issues
- Manage firewalls, ports, and network services
- Set up virtual networks (important in Docker/Kubernetes)

FOLLOW THE PACKET - what to run at each hop

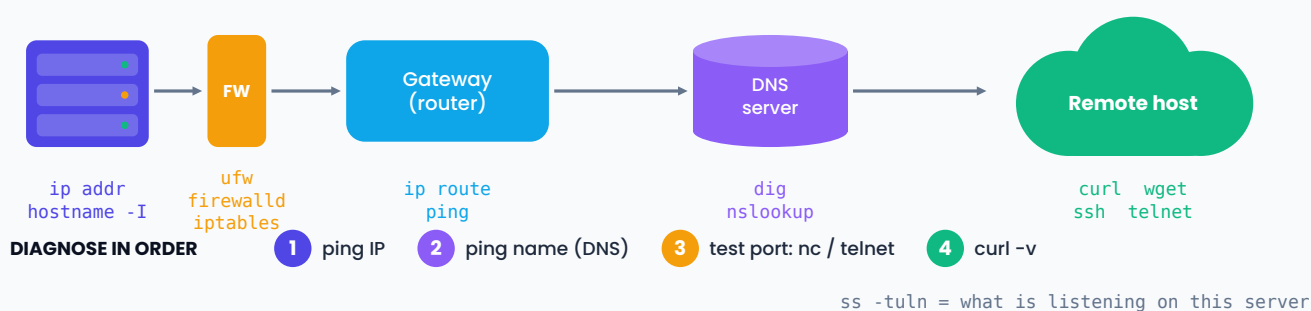


Fig 14. Which tool to use at each step between your server and the internet.

1. Essential networking commands

Task	Command
Check IP address & interfaces	<code>ip addr show</code> (modern), <code>ifconfig</code> (legacy, needs net-tools), <code>hostname -I</code> (IP only)
View routing table	<code>ip route show</code> , <code>route -n</code>
Check DNS info	<code>cat /etc/resolv.conf</code>
View / change hostname	<code>hostname</code> (show), <code>hostnamectl set-hostname newname</code>

2. Connectivity & troubleshooting tools

Tool	Use
<code>ping <host></code>	Check if host is reachable
<code>traceroute <host></code>	Show path to host
<code>nslookup <host></code>	DNS resolution
<code>dig <host></code>	Advanced DNS info
<code>telnet <host> <port></code>	Test port connectivity
<code>nmap <host></code>	Scan open ports
<code>netstat -tulnp</code>	Show listening ports (TCP/UDP)
<code>ss -tuln</code>	Modern version of netstat
<code>curl</code> , <code>wget</code>	Test HTTP/HTTPS connections
<code>tcpdump</code>	Packet capture tool

Tool	Use
ip a, ip r, ip link	Interface and routing config

3. Configure network interfaces

Using ip (modern tool)

Assign IP manually:

```
sudo ip addr add 192.168.1.10/24 dev eth0
sudo ip link set eth0 up
```

Remove IP:

```
sudo ip addr del 192.168.1.10/24 dev eth0
```

Using nmcli (NetworkManager CLI, GUI-based systems)

```
nmcli device status          # Show devices
nmcli connection show       # List connections
nmcli connection up "Wired connection 1" # Enable connection
```

4. Network configuration files

File	Purpose
/etc/hosts	Map hostnames to IPs manually
/etc/hostname	System's hostname
/etc/resolv.conf	DNS nameservers
/etc/network/interfaces	(Debian-based static IP config)
/etc/sysconfig/network-scripts/ifcfg-*	(RHEL-based network config)

5. Firewall management

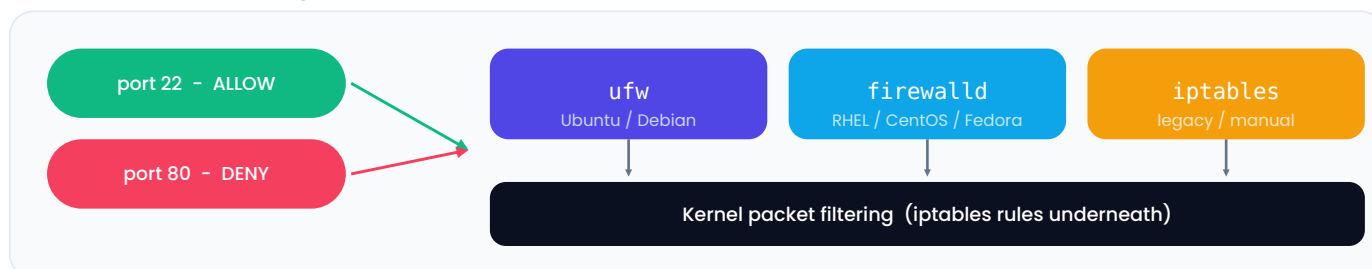


Fig 15. Firewall tools by distro. All filter packets in the kernel.

UFW (Ubuntu/Debian)

```
sudo ufw status
sudo ufw allow 22    # Allow SSH
sudo ufw deny 80
sudo ufw enable
```

Firewalld (RHEL/CentOS/Fedora)

```
sudo firewall-cmd --list-all
sudo firewall-cmd --add-port=8080/tcp --permanent
sudo firewall-cmd --reload
```

iptables (legacy/manual)

```
sudo iptables -L    # List rules
sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT
```

Many DevOps tools still use iptables rules underneath.

6. Host to host communication

Task	Command
Test SSH	<code>ssh user@host</code>
Share file	<code>scp file.txt user@host:/path</code>
Sync folders	<code>rsync -av /source/ user@host:/target/</code>
Setup port forwarding	<code>ssh -L 8080:localhost:80 user@remote</code>

7. DevOps real-world use cases

Scenario	Tool / command
Check server port before deploy	<code>ss -tuln grep <port></code>
Validate DNS for API call	<code>dig api.myapp.com</code>
Configure static IP on cloud VM	<code>/etc/netplan/*.yaml</code> or <code>nmcli</code>
Expose container port to host	<code>docker run -p 8080:80 nginx</code>
Detect broken microservice	<code>curl http://service:port/health</code>
Monitor traffic	<code>iftop</code> , <code>nethogs</code> , <code>tcpdump</code>
Test firewall blocking	<code>telnet host port</code> or <code>nmap host</code>

Summary table

Tool	Purpose
<code>ip</code>	Modern IP and route management
<code>ss</code> , <code>netstat</code>	View listening services
<code>ping</code> , <code>traceroute</code>	Network diagnostics
<code>curl</code> , <code>wget</code>	Test HTTP/HTTPS
<code>dig</code> , <code>nslookup</code>	DNS tools
<code>nmap</code>	Port scanner
<code>iptables</code> , <code>ufw</code> , <code>firewalld</code>	Firewalls
<code>nmcli</code> , <code>nmtui</code>	GUI/CLI network config

Practice task

```
# Check IP and routes
ip a
ip route
# Ping gateway
ping 192.168.1.1
# Test DNS resolution
dig google.com
# Scan open ports
nmap localhost
# Check service listening
ss -tuln | grep 22
# Allow HTTP in firewall
sudo ufw allow 80
```

11

Troubleshooting

Find and fix problems step by step



What is troubleshooting?

Troubleshooting in Linux means identifying and fixing:

- System errors
- Service failures
- Performance bottlenecks
- Network issues
- Disk problems
- Permissions and access issues

1. Basic diagnostic questions

Before diving into tools, ask:

- What exactly isn't working?
- When did it start?
- Has anything changed recently (updates, config, deployments)?
- Can it be replicated?
- Logs available?

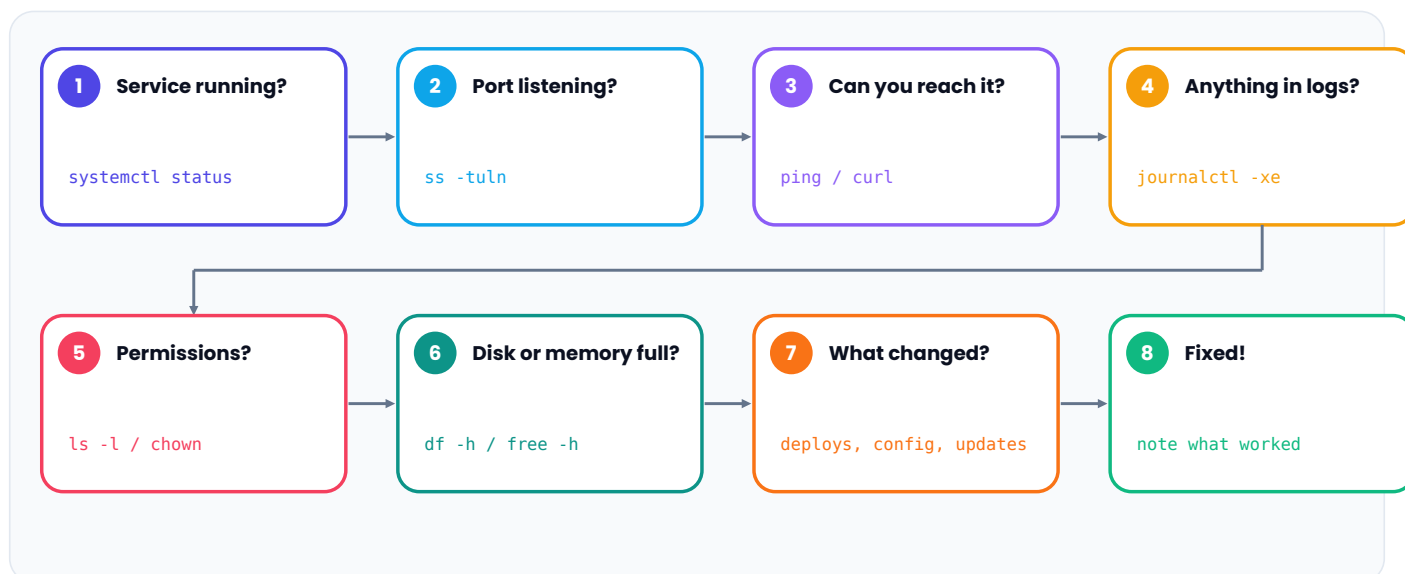


Fig 16. The troubleshooting checklist as a flow.

2. System health check

CPU, memory, load

```
top
htop
uptime      # Check load average
free -h     # Check memory usage
vmstat 1    # System performance in real-time
```

Disk usage

```
df -h      # Show available disk space
du -sh *   # Check size of subdirectories
lsblk      # List block devices
```

Check disk errors

```
dmesg | grep -i error
sudo smartctl -a /dev/sda
```

3. Network troubleshooting

Task	Command
Check IP address	<code>ip a</code> or <code>ifconfig</code>
Ping a host	<code>ping 8.8.8.8</code> or <code>ping google.com</code>
Trace route	<code>traceroute google.com</code>
DNS check	<code>dig</code> , <code>nslookup</code>
Port test	<code>telnet host port</code> , <code>nmap</code> , or <code>nc -zv host port</code>
View open/listening ports	<code>ss -tuln</code> , <code>netstat -tulnp</code>
Restart network	<code>systemctl restart NetworkManager</code> or <code>sudo service networking restart</code>

4. Log files (goldmine of info)

Location	Purpose
<code>/var/log/syslog</code> or <code>/var/log/messages</code>	General system logs
<code>/var/log/auth.log</code>	Login attempts, sudo access
<code>/var/log/dmesg</code>	Kernel and hardware logs
<code>/var/log/nginx/</code> or <code>/var/log/httpd/</code>	Web server logs
<code>/var/log/mysql/</code>	DB logs
<code>/var/log/journal/</code>	systemd logs (if journald enabled)

View logs:

```
tail -n 50 /var/log/syslog
journalctl -xe
```

5. Service failures (systemd-based systems)

Check status and logs:

```
systemctl status nginx
journalctl -u nginx -b
```

Restart or reload:

```
sudo systemctl restart nginx
sudo systemctl daemon-reload
```

6. Permission issues

Common symptoms:

- **Permission denied** errors
- Files not executable
- Services unable to read config files

Commands:

```
ls -l filename
chmod +x script.sh
chown user:group file
```

Check if the service is running as the correct user (e.g., nginx, www-data).

7. Application debugging (e.g., Python, Bash, Node.js)

Tool	Use
<code>bash -x script.sh</code>	Debug bash scripts line by line
<code>python3 -m pdb script.py</code>	Python debugger
<code>npm run dev</code> or <code>node app.js</code>	Watch console logs for Node.js
<code>curl -v</code>	Verbose API call debugging

8. Database troubleshooting

Action	Command
Check MySQL status	<code>systemctl status mysql</code>
Access MySQL	<code>mysql -u root -p</code>
View DB logs	<code>/var/log/mysql/error.log</code>
PostgreSQL logs	<code>/var/log/postgresql/postgresql.log</code>
Test DB connection	<code>mysqladmin ping</code> or <code>psql -h host -U user</code>

9. Package / dependency problems

Debian/Ubuntu:

```
sudo apt update
sudo apt install -f
sudo dpkg --configure -a
```

RHEL/CentOS:

```
sudo yum clean all
sudo yum check
```

10. Process & resource issues

Task	Command
List processes	<code>ps -ef, top, htop</code>
Kill process	<code>kill <PID> or pkill <name></code>
Zombie process check	<code>ps aux grep Z</code>
Memory hogs	<code>ps aux --sort=-%mem head</code>
CPU hogs	<code>ps aux --sort=-%cpu head</code>

11. Reboot, shutdown, recovery

Task	Command
Reboot	<code>sudo reboot</code>
Shutdown	<code>sudo shutdown now</code>
Rescue mode (grub)	Hold Shift at boot, select Advanced, then Recovery mode
Remount root fs	<code>mount -o remount,rw /</code>
Filesystem check	<code>fsck /dev/sda1</code> (carefully, needs unmounted disk)

12. Troubleshooting checklist

- Is the service running?
- Is the port open and listening?
- Can you ping / curl the target?
- Do you see anything in the logs?
- Is it a permission or ownership problem?
- Is the disk or memory full?
- Has anything changed (deployment, config, updates)?

Tools worth installing

Tool	Description
<code>htop</code>	Advanced resource monitor
<code>iftop, nethogs</code>	Live network usage
<code>ncdu</code>	Disk usage analyzer
<code>strace</code>	Trace system calls
<code>lsof</code>	List open files
<code>tcpdump</code>	Network packet capture

Sample scenario to practice

```
# Your service stopped working
systemctl status myapp.service
journalctl -u myapp -n 50
ss -tuln | grep 8080
df -h
ps -ef | grep myapp

# Step 1: Check status
# Step 2: Check logs
# Step 3: Port listening?
# Step 4: Disk space?
# Step 5: Process alive?
```

12

Practical: User Access Control

Full sudo vs limited command access

Scenario: you are a DevOps/Linux admin and need to add a new user and give them either **full sudo (admin) access** or **limited command access** (e.g., restart Apache only). With clear examples, commands, and explanations for each step.

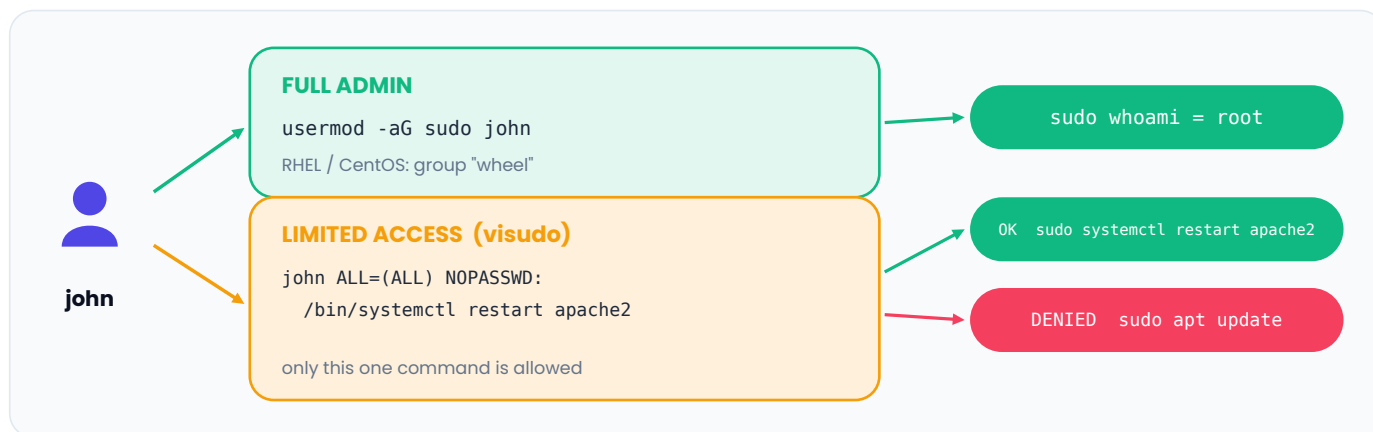


Fig 17. Two ways to give a user access: full or limited.

Step-by-step: create and manage Linux user access

Step 1: Create a new user

```
sudo adduser john
```

This creates a new user **john** with a home directory.

Step 2: Set a password for the user

```
sudo passwd john
```

A prompt will ask for and confirm a password.

Step 3: Give full admin (sudo) access

Option	Command
Option 1: add to the sudo group (Debian/Ubuntu)	<code>sudo usermod -aG sudo john</code>
Option 2: for CentOS/RHEL	<code>sudo usermod -aG wheel john</code>

This gives full root-equivalent access via sudo. **Test:** login as john and run `sudo whoami`. Expected output: **root**.

Partial / limited access (for security or compliance)

Step 4: Allow a specific command via sudoers

Use the `visudo` command to safely edit the sudoers file:

```
sudo visudo
```

Inside the file, add the rule below at the end:

```
john ALL=(ALL) NOPASSWD: /bin/systemctl restart apache2
```

Part	Explanation
john	The username
ALL=(ALL)	Can run as any user

Part	Explanation
NOPASSWD	Won't be prompted for password
/bin/systemctl restart apache2	Only this command allowed with sudo

Test: login as john, then run:

```
sudo /bin/systemctl restart apache2 # Should work (OK)
sudo apt update                    # Permission denied (X)
```

Bonus: allow multiple commands

```
john ALL=(ALL) NOPASSWD: /sbin/ifconfig, /usr/bin/apt-get update
```

How to find command full paths

```
which <command>
which systemctl
```

Restrict shell access (optional)

To block shell login but allow cron/FTP:

```
sudo usermod -s /usr/sbin/nologin john
```

Check group of a user

```
groups john
```

Remove a user's sudo access

Remove user from sudo group:

```
sudo deluser john sudo # Ubuntu/Debian
sudo gpasswd -d john wheel # RHEL/CentOS
```

Or remove the entry from visudo.

Summary table

Task	Command
Add user	<code>sudo adduser john</code>
Set password	<code>sudo passwd john</code>
Full admin	<code>sudo usermod -aG sudo john</code>
Partial access	<code>sudo visudo</code> then add custom rule
Check access	<code>sudo -l -U john</code>
Remove sudo	<code>sudo deluser john sudo</code>

Real-world example use case (DevOps)

TRY IT

Use case: give a DevOps intern permission only to restart Nginx and view logs.

1. Create user:

```
sudo adduser devopsintern
```

2. Edit sudoers via visudo:

```
devopsintern ALL=(ALL) NOPASSWD: /bin/systemctl restart nginx, /usr/bin/tail -n 100 /var/log/nginx/error.log
```

Now they can:

```
sudo /bin/systemctl restart nginx  
sudo /usr/bin/tail -n 100 /var/log/nginx/error.log
```

But they **cannot do anything else** with sudo.

13

File System Structure

Everything starts at /



The Linux file system is a **hierarchical directory structure** that starts from the root (/) directory and branches out to other subdirectories.

1. Root directory /

- The top-level directory of the Linux file system.
- All files and directories start from here.
- It is the parent of all other directories.

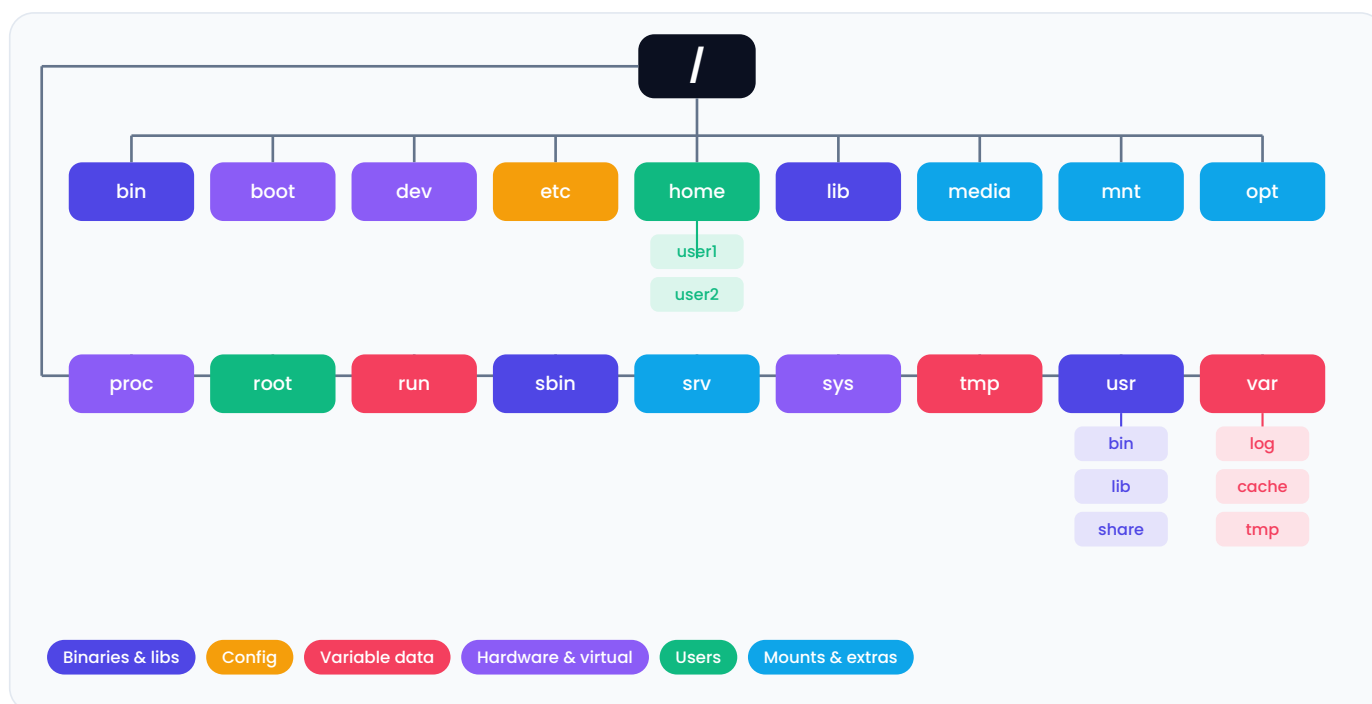


Fig 18. The Linux file system hierarchy, colour-coded by purpose.

2. Standard directories under /

Directory	Description
/bin	Essential user binaries (commands): ls, cp, mv, rm, etc.
/sbin	System binaries: admin-level tools like iptables, reboot, ifconfig.
/boot	Bootloader files, Linux kernel (vmlinuz), initrd, GRUB config.
/dev	Device files: e.g., /dev/sda, /dev/tty0. Represents hardware devices as files.
/etc	System-wide configuration files: e.g., /etc/passwd, /etc/hosts, /etc/fstab.
/home	User home directories: /home/riyas, /home/mohan.
/lib	Essential shared libraries for binaries in /bin and /sbin.
/media	Mount point for removable media: USB, CD/DVD auto-mounted here.
/mnt	Used for temporarily mounting file systems manually by the user.
/opt	Optional application packages (e.g., third-party apps like Chrome, VMware).
/proc	Virtual filesystem providing process and kernel info (e.g., /proc/cpuinfo).

Directory	Description
<code>/root</code>	Home directory of root user. Different from <code>/home/root</code> .
<code>/run</code>	Runtime data since the last boot, like PID files, socket files.
<code>/srv</code>	Service-related data for servers like FTP, WWW.
<code>/sys</code>	Virtual filesystem showing kernel info about devices.
<code>/tmp</code>	Temporary files. Cleared on reboot.
<code>/usr</code>	Secondary hierarchy for read-only user data; has subdirs like <code>/usr/bin</code> , <code>/usr/lib</code> .
<code>/var</code>	Variable data like logs, spool files, cache, and emails.

3. Important `/etc` files and their use

File	Purpose
<code>/etc/passwd</code>	Stores user account information
<code>/etc/shadow</code>	Contains user password hashes
<code>/etc/group</code>	Stores group info
<code>/etc/hostname</code>	Sets system hostname
<code>/etc/fstab</code>	Contains auto-mount information
<code>/etc/hosts</code>	Maps IP addresses to hostnames
<code>/etc/resolv.conf</code>	DNS configuration
<code>/etc/systemd/</code>	Contains service files and system boot configuration

4. Important `/var` subdirectories

Directory	Purpose
<code>/var/log</code>	System log files (important for troubleshooting)
<code>/var/spool</code>	Print and mail spools
<code>/var/tmp</code>	Temp files preserved between reboots
<code>/var/cache</code>	Cached data from applications
<code>/var/www</code>	Web files (for Apache/Nginx servers)

5. Virtual filesystems: `/proc`, `/sys`, `/dev`



Fig 19. Virtual filesystems expose the system as files.

6. Package & binary locations

Path	Use
<code>/bin</code>	Essential system binaries
<code>/usr/bin</code>	Non-essential user commands
<code>/sbin, /usr/sbin</code>	Admin commands
<code>/lib, /usr/lib</code>	Shared libraries
<code>/opt</code>	Third-party packages

7. User & permissions

- Each user has a home directory inside **/home**.
- The root user's home is **/root**.
- Use `ls -l /` to view permissions and ownership of top-level directories.

8. Mount points

Devices or partitions are attached to the Linux file system via **mount points**. Example: `mount /dev/sdb1 /mnt/usb`. Common mount paths are `/mnt`, `/media`, `/run/media`.

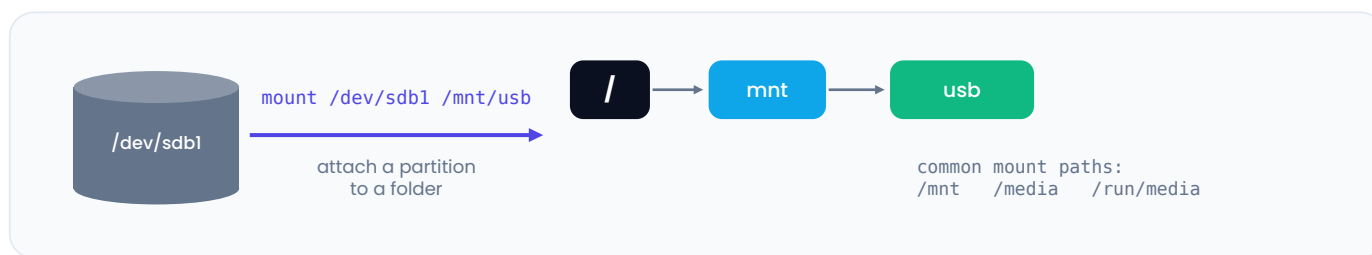


Fig 20. Mounting a partition onto a folder.

9. Key concepts to remember

- **Everything in Linux is a file** (including hardware and processes).
- The file system follows **FHS** (Filesystem Hierarchy Standard).
- Root (`/`) is the origin: all other directories are children of it.
- **Separation of concerns**: logs, binaries, configs, libraries all have dedicated locations.

Diagram of Linux file system hierarchy

```
/
├── bin
├── boot
├── dev
├── etc
├── home
│   ├── user1
│   └── user2
├── lib
├── media
├── mnt
├── opt
├── proc
├── root
├── run
├── sbin
├── srv
├── sys
├── tmp
├── usr
│   ├── bin
│   ├── lib
│   └── share
└── var
    ├── log
    ├── cache
    └── tmp
```